

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа № 3

Выполнил:

Бессонов Александр

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Мигрировать ранее разработанное веб-приложение “Т-Пульс” на Vue.js и оформить его как одностраничное приложение. Подключить клиентскую маршрутизацию, реализовать работу с внешним REST API посредством Axios, выполнить разумное деление интерфейса на компоненты и вынести повторяющуюся логику в composable-функции.

Ход работы

- 1. Подготовка проекта.** За основу взят интерфейс, разработанный в предыдущих работах. Проект переведён на Vue 3 и Vite. Установлены зависимости vue-router, axios, Pinia, json-server, json-server-auth и concurrently. Команда `npm start` одновременно запускает API и клиентскую часть.
- 2. Клиентская маршрутизация.** Созданы маршруты входа, регистрации, обзора, проекта, бэклога, команды и поиска. Метаданные маршрутов определяют заголовок страницы и требования к авторизации. Глобальная `navigation guard` перенаправляет неавторизованного пользователя на страницу входа.
- 3. Работа с API и состоянием.** Единый экземпляр Axios обращается к REST API через префикс `/api`, добавляет Bearer-токен и обрабатывает недоступность сервера. Данные пользователя, задач, проектов, участников и уведомлений распределены по Pinia-хранилищам. Операции создания и изменения задач выполняются через API.
- 4. Компонентный подход.** Повторяющиеся части интерфейса выделены в компоненты оболочки, боковой и верхней навигации, карточек и списков задач, модального окна и панели уведомлений. Страницы содержат только композицию компонентов и относящуюся к ним логику представления.
- 5. Composable-функции.** Логика переключения и сохранения темы вынесена в `useTheme`, а фильтрация задач и сброс строки поиска — в `useTaskFilters`. Composable повторно используются в нескольких представлениях.
- 6. Проверка результата.** Проект успешно собирается командой `npm run build`. Проверены все маршруты, вход и выход, защита закрытых страниц, поиск и фильтры, отображение данных API, светлая и тёмная темы, уведомления и модальное создание задачи.

Основные команды npm

<code>npm install</code>	# установка зависимостей
<code>npm start</code>	# параллельный запуск REST API и Vite
<code>npm run build</code>	# production-сборка приложения
<code>npm run preview</code>	# локальная проверка production-сборки

Листинг 1 - Сценарии запуска и сборки проекта

Маршрутизация и navigation guards

Vue Router работает в режиме HTML5 History. Каждый экран имеет собственный URL и заголовок. Метаданные `requiresAuth` и `guestOnly` используются глобальной защитой маршрутов: закрытые страницы недоступны без токена, а авторизованный пользователь не возвращается на формы входа и регистрации.

```
const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', redirect: '/dashboard' },
    { path: '/login', component: LoginView,
      meta: { title: 'Вход', guestOnly: true } },
    { path: '/dashboard', component: DashboardView,
      meta: { requiresAuth: true, title: 'Обзор' } },
    { path: '/project', component: ProjectView,
      meta: { requiresAuth: true, title: 'Проект' } },
    { path: '/backlog', component: BacklogView,
      meta: { requiresAuth: true, title: 'Бэклог' } },
    { path: '/team', component: TeamView,
      meta: { requiresAuth: true, title: 'Команда' } },
    { path: '/search', component: SearchView,
      meta: { requiresAuth: true, title: 'Поиск' } },
  ],
})

router.beforeEach((to) => {
  const authenticated = hasSession()
  if (to.meta.requiresAuth && !authenticated)
    return { name: 'login', query: { redirect: to.fullPath } }
  if (to.meta.guestOnly && authenticated)
    return { name: 'dashboard' }
  return true
})
```

Листинг 2 - Конфигурация маршрутов и глобальная navigation guard

Внешний REST API средствами Axios

Для всех HTTP-запросов создан общий клиент Axios. Vite проксирует путь `/api` на локальный REST-сервер, поэтому компоненты не зависят от конкретного порта API. Request interceptor автоматически добавляет токен авторизации, а response interceptor формирует понятное сообщение при отсутствии соединения.

```
export const http = axios.create({
  baseURL: '/api',
  timeout: 5000,
})

http.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY)
  || sessionStorage.getItem(TOKEN_KEY)
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

http.interceptors.response.use(
  (response) => response,
  (error) => {
    if (!error.response)
      return Promise.reject(new Error('API недоступен'))
    return Promise.reject(error)
  },
)
```

Листинг 3 - Общий HTTP-клиент Axios

Компонентная архитектура

Приложение разделено на представления маршрутов, общие компоненты интерфейса, Pinia-хранилища, API-модули и composable-функции. Такое устройство уменьшает связанность кода и позволяет повторно использовать навигацию, карточки задач, фильтры и диалоговые элементы.

```
App.vue
RouterView
views/
  LoginView, RegisterView, DashboardView
  ProjectView, BacklogView, TeamView, SearchView
components/
  WorkspaceShell -> AppSidebar + AppTopbar
  KanbanColumn + TaskListItem
  NotificationsDrawer + NewTaskModal
stores/ -> auth, tasks, team
composables/ -> useTheme, useTaskFilters
api/ -> единый Axios-клиент
```

Листинг 4 - Логическая структура Vue-приложения

Назначение основных компонентов

Компонент	Назначение
WorkspaceShell	Единая оболочка защищённых страниц и область RouterView.
AppSidebar, AppTopbar	Навигация, активное состояние маршрута, тема и действия пользователя.
KanbanColumn	Колонка доски с задачами одного статуса.
TaskListItem	Строковое представление задачи для бэклога и поиска.
NotificationsDrawer	Боковая панель уведомлений и отметка сообщений прочитанными.
NewTaskModal	Форма создания задачи с передачей события родительскому компоненту.

Composable для повторяющейся логики

Функция `useTaskFilters` инкапсулирует реактивную строку поиска, вычисляемый список и операцию сброса. Её можно применять на разных страницах, передавая исходный массив задач как `ref` или `computed`.

```
export function useTaskFilters(tasks) {
  const searchQuery = ref('')
  const filteredTasks = computed(() => {
    const query = searchQuery.value.trim().toLocaleLowerCase('ru-RU')
    if (!query) return toValue(tasks)
    return toValue(tasks).filter((task) =>
      [task.title, task.project, task.priority]
        .join(' ').toLocaleLowerCase('ru-RU').includes(query),
    )
  })
  const resetSearch = () => { searchQuery.value = '' }
  return { searchQuery, filteredTasks, resetSearch }
}
```

Листинг 5 - Composable-функция `useTaskFilters`

Проверка интерфейса: проект и бэклог

После авторизации доступны маршруты рабочего пространства. Страница проекта получает задачи из REST API, распределяет их по статусам и отображает на канбан-доске. Бэклог использует общий список задач и собственные реактивные фильтры.

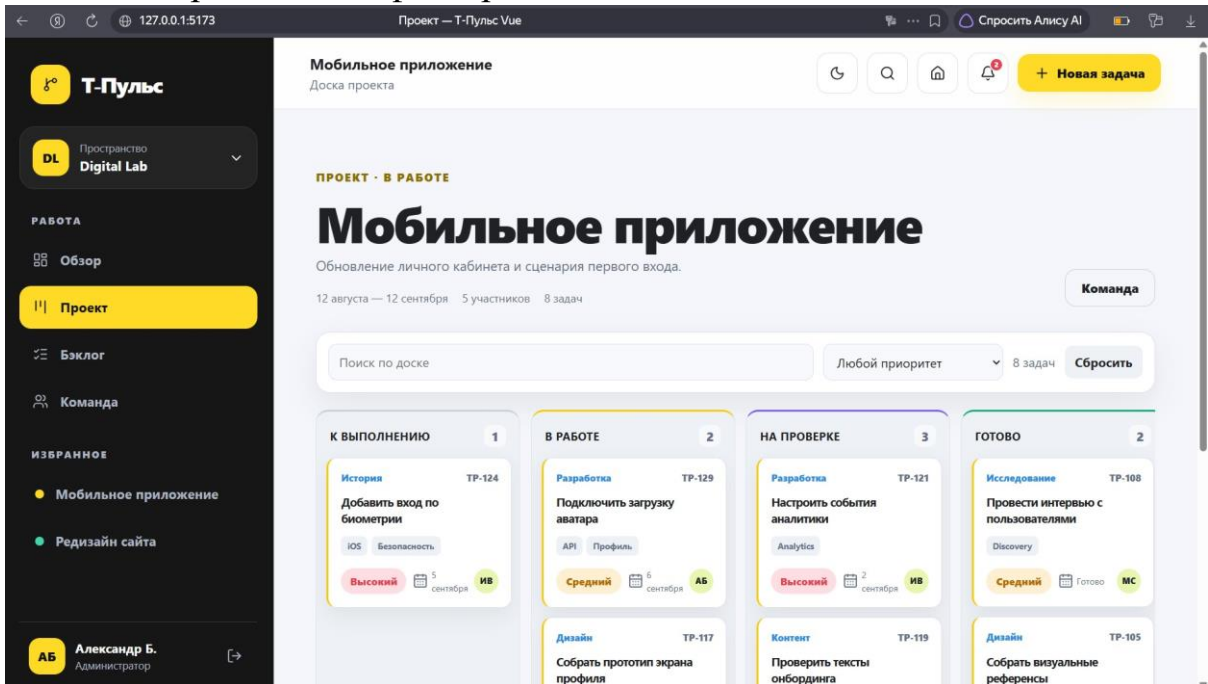


Рисунок 1 - Доска проекта с задачами, полученными из REST API

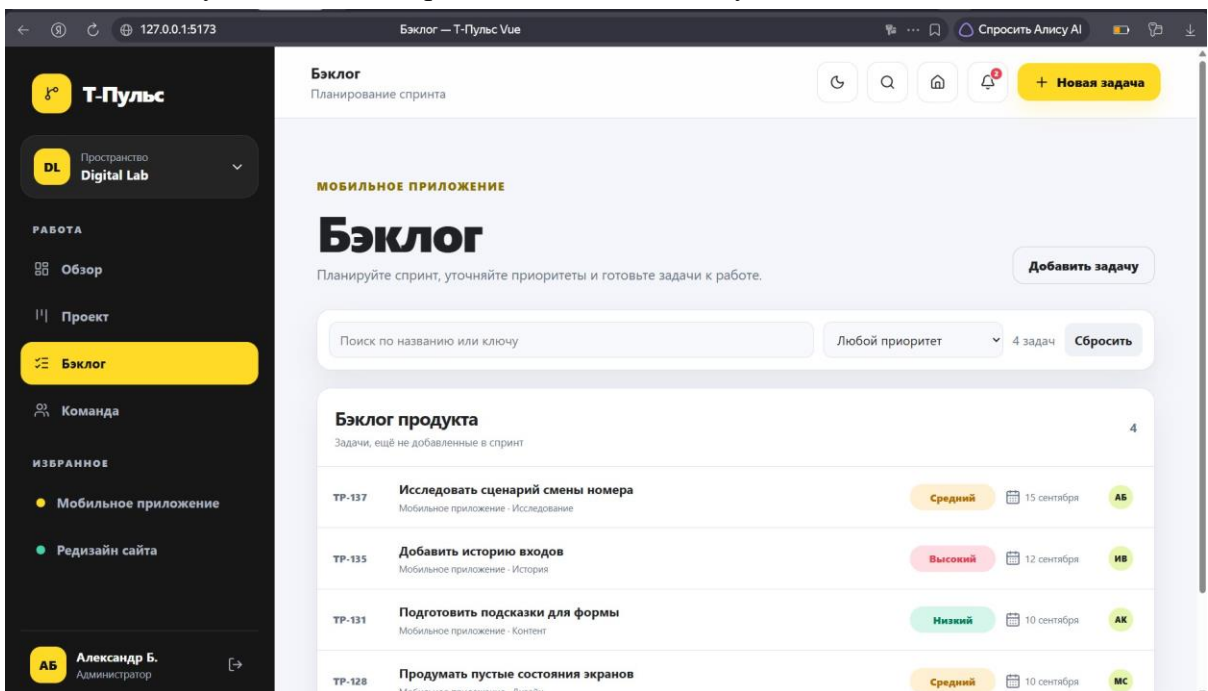


Рисунок 2 - Страница бэклога с поиском и фильтрацией

Проверка интерфейса: команда и поиск

Отдельные маршруты демонстрируют повторное использование общей оболочки приложения. Раздел команды отображает участников рабочего пространства, а глобальный поиск фильтрует задачи по названию, проекту, приоритету и разделу.

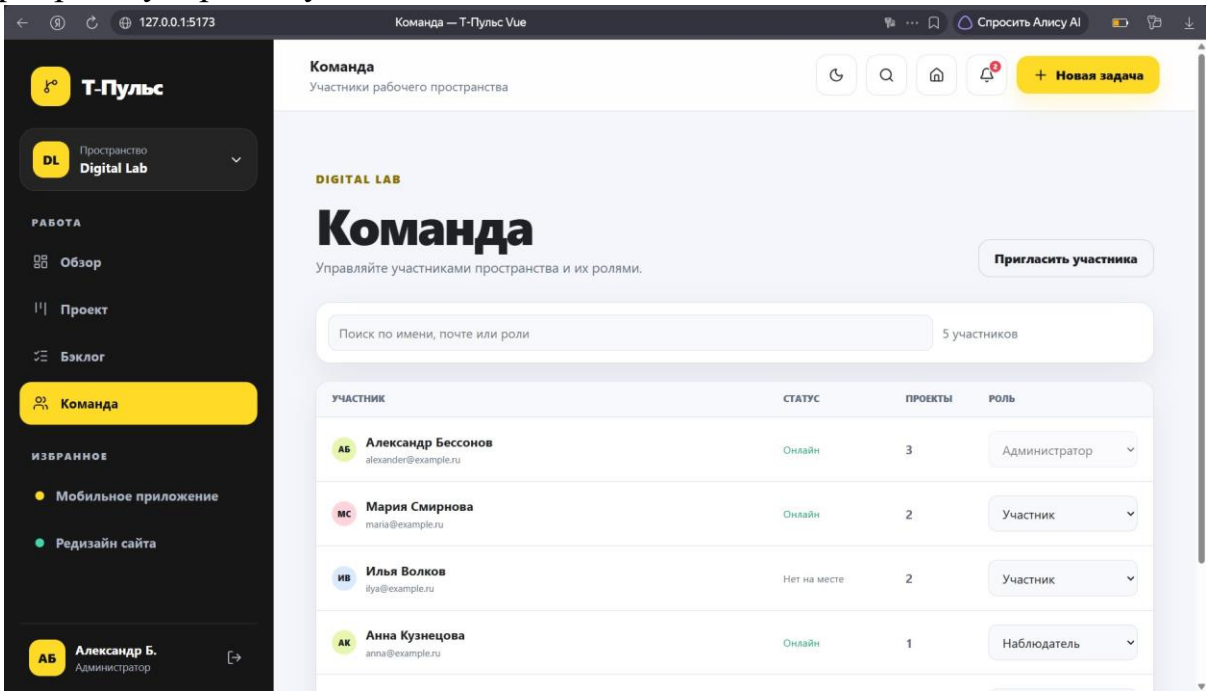


Рисунок 3 - Страница команды с данными участников

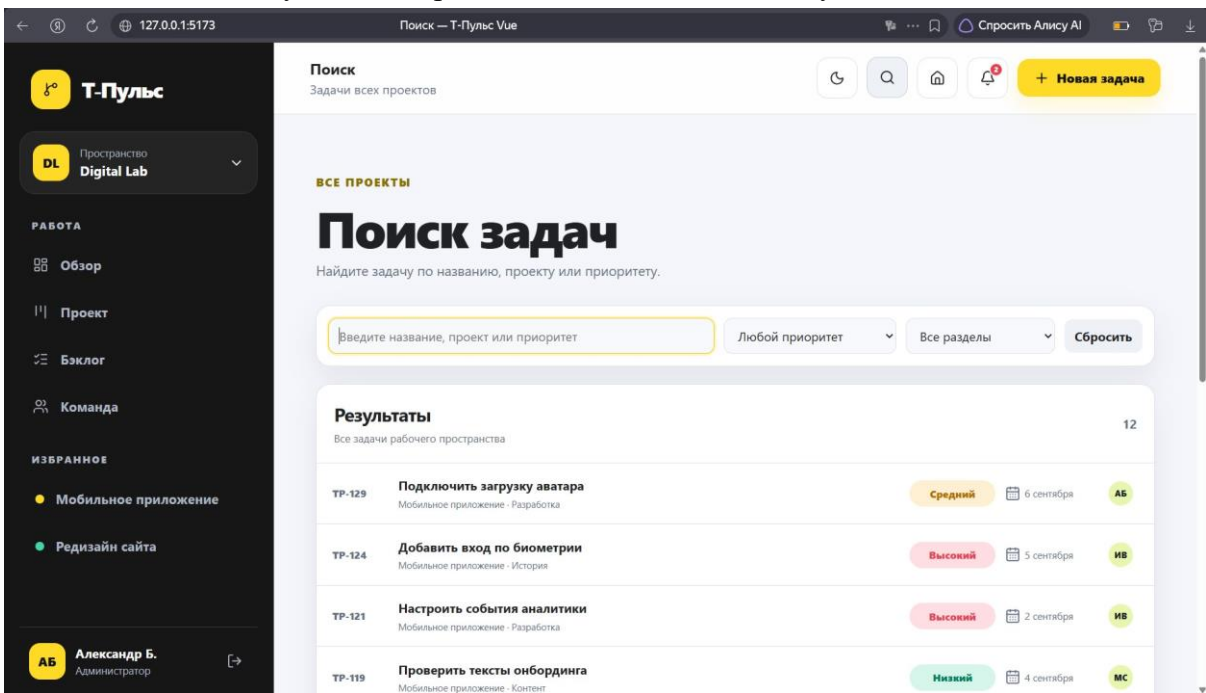


Рисунок 4 - Глобальный поиск задач в светлой теме

Тема и уведомления

Composable useTheme переключает светлую и тёмную темы, записывает выбор в localStorage и восстанавливает его при следующем запуске. Панель уведомлений реализована как отдельный компонент и открывается поверх текущего маршрута.

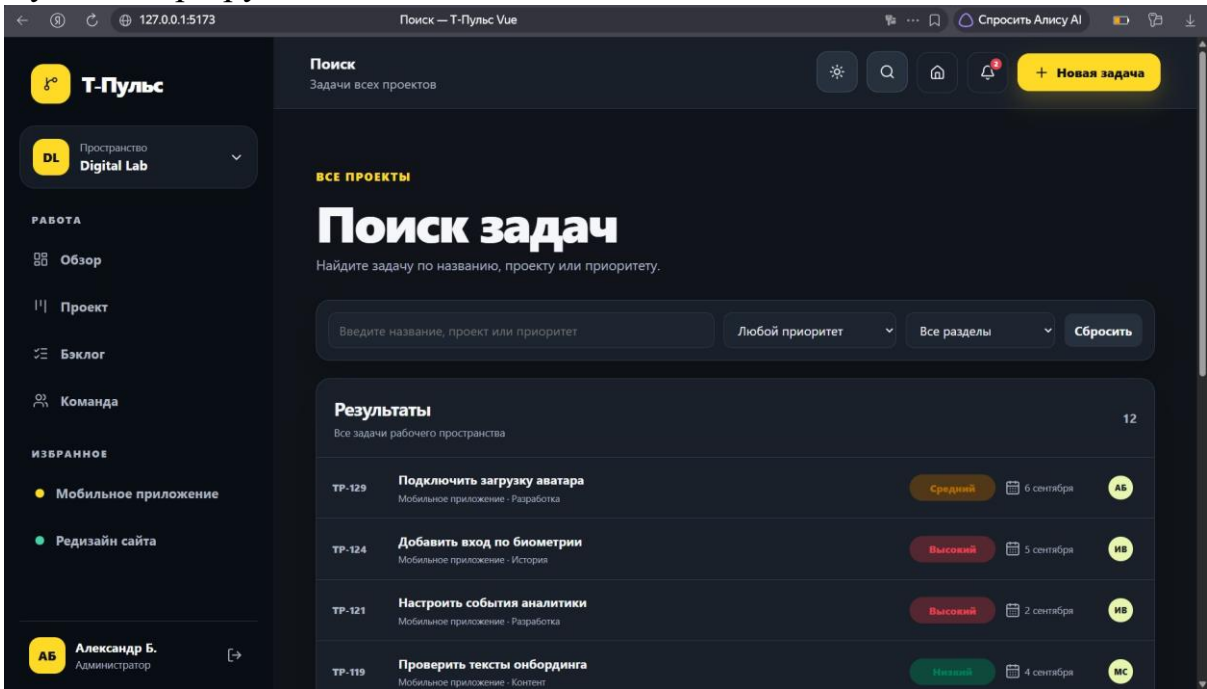


Рисунок 5 - Страница поиска в тёмной теме

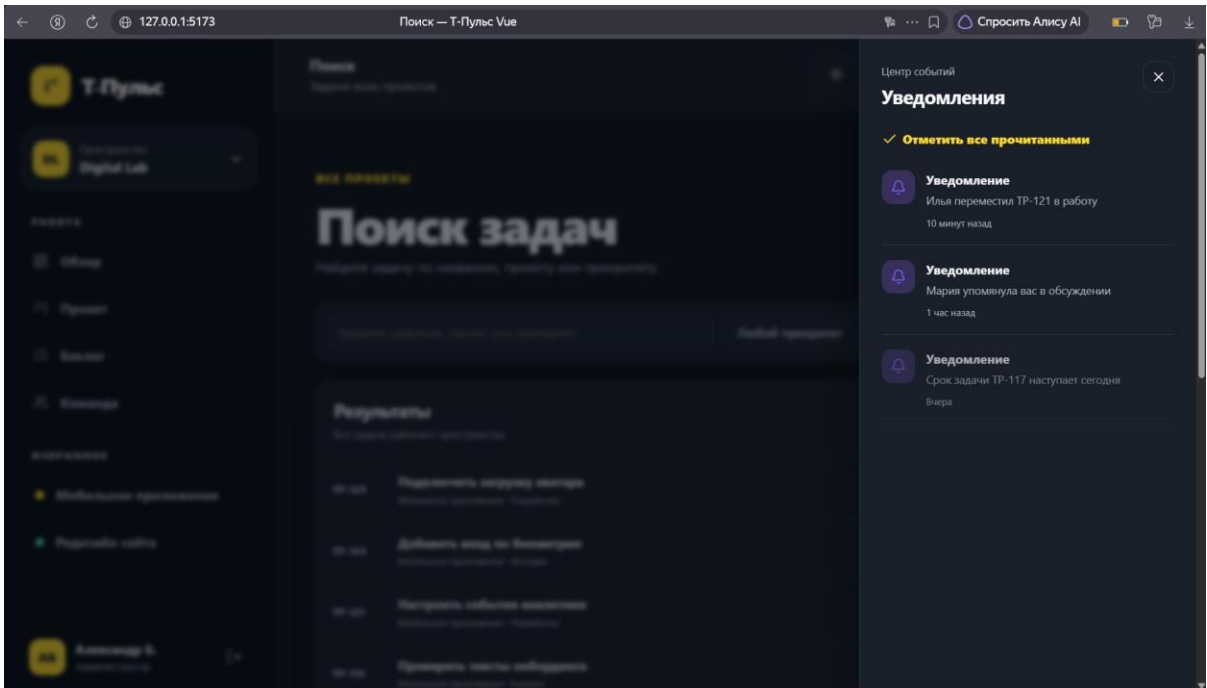


Рисунок 6 - Компонент панели уведомлений

Диалоговые элементы и авторизация

Модальное окно создания задачи является переиспользуемым компонентом с управляемым состоянием, валидацией формы и событием create. Форма входа работает с хранилищем авторизации и Axios API; пароль не сохраняется, а закрытые маршруты защищены navigation guard.

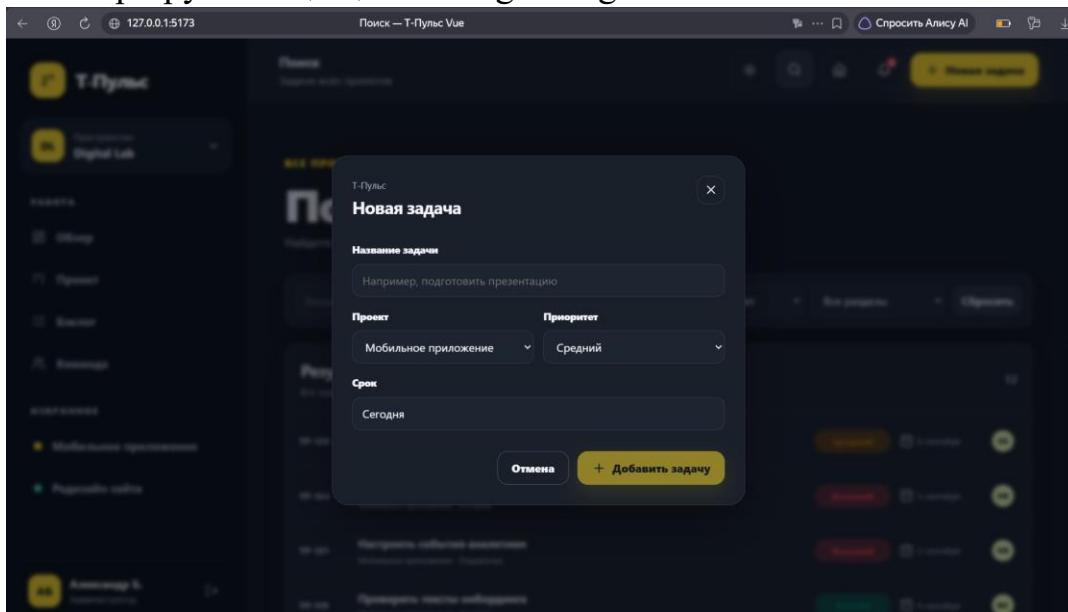


Рисунок 7 - Модальное окно создания новой задачи

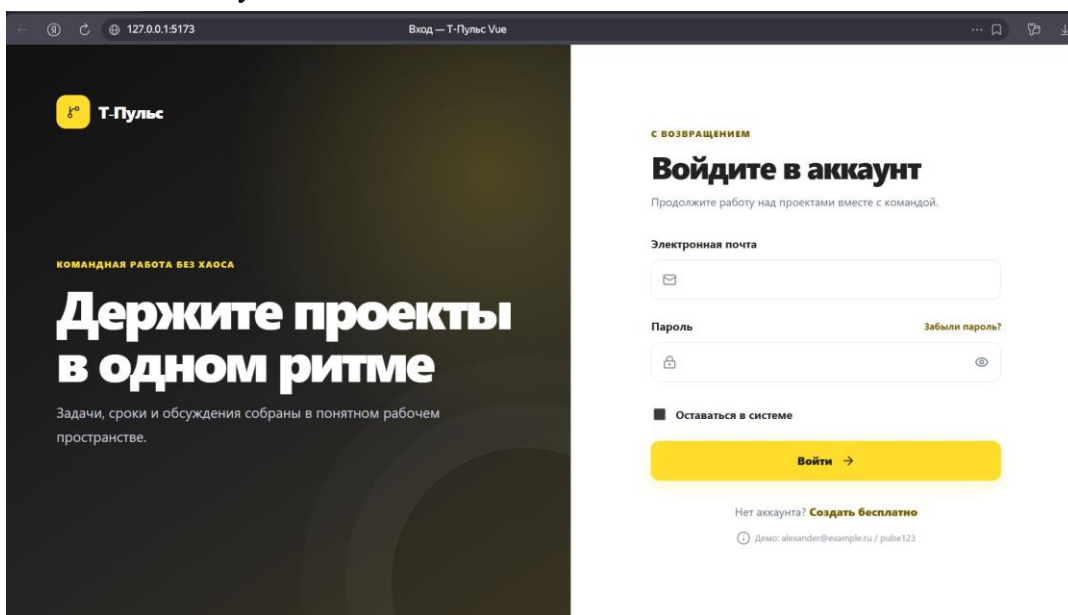


Рисунок 8 - Страница входа с очищенными полями

Вывод

В ходе лабораторной работы ранее разработанное приложение “Т-Пульс” мигрировано на Vue.js и оформлено как полноценное SPA. Подключены Vue Router с метаданными и navigation guards, Pinia для управления состоянием и Axios для работы с внешним REST API. Интерфейс разделён на переиспользуемые компоненты, а общая реактивная логика темы и фильтрации вынесена в composable-функции. Сборка и основные пользовательские сценарии успешно проверены.